**AIR UNIVERSITY ISLAMABAD CAMPUS**
**VISUAL PROGRAMMING (CS-284)**
**SPRING 2026**
**ASSIGNMENT-2**
**CLO- 3 GA-5**

---

**Due Date: April 24, 2026 (11:59 PM)**

**Marks: 100**

**Instructions:**

1. *Make sure that you read and understand each and every instruction.*
2. *It is a group assignment having a group size of **3 members**. **No more or less than 3 members are allowed in any group.***
3. *Plagiarism is strongly forbidden and will be very strongly punished. If we find that you have copied from someone else or someone else has copied from you (with or without your knowledge) both of you will be punished. You will be awarded straight zero in this assignment.*
4. *Submit a single '.zip' file for your assignment that will have separate .razor/.cs for each class or component. **Rename the submission with the roll numbers of every member of the group.***

# EduConnect: A University Academic Web Portal

## 1. Project Overview

EduConnect is a comprehensive, fully functional university academic portal built using Blazor and C#. The system simulates a real-world academic management platform that universities use to manage students, courses, faculty, grades, and notifications. This assignment is deliberately designed to be ambitious yet achievable — it will push you to apply every concept taught from Week 1 through Week 10 of the course, in a cohesive, professional-grade application.

Unlike toy exercises, EduConnect is a living, breathing system with multiple interconnected modules. Every design decision you make must be grounded in the principles you have studied: object-oriented design, event-driven programming, SOLID architecture, C# advanced features, and reactive Blazor UI through data binding. You will not simply build screens — you will engineer a system.

### 1.1 The Problem Being Solved

Modern universities manage thousands of students, hundreds of courses, and dozens of faculty members. Historically, this is done through fragmented systems — spreadsheets, emails, and paper forms. EduConnect replaces this chaos with a unified, role-based, event-driven academic portal. Your implementation must handle the following real-world concerns:

- Multiple user roles: Admin, Faculty, and Student — each with different views and permissions
- Real-time UI updates when data changes (Blazor two-way data binding)
- A well-structured C# class hierarchy
- Notification events that fire when grade updates, enrollment changes, or deadlines occur
- Robust error handling for invalid inputs and edge cases
- A clean, Bootstrap-styled responsive interface that degrades gracefully on mobile

### 1.2 Technology Stack

- Frontend Framework: Blazor (.NET 8)
- UI Styling: Bootstrap 5 with custom CSS (no third-party component libraries)
- Language: C# (.NET 8)
- Data Layer: In-memory repository pattern (no database required — simulated with C# collections)
- State Management: Scoped services with event-driven notifications
- IDE: Visual Studio 2022

## 2. Functional Requirements-Modules in Detail

EduConnect is composed of five major modules. Each module is a mandatory part of the submission. Skipping a module result in the loss of all marks allocated to that module. The following sections describe each module in precise detail.

### 2.1 Module 1: Authentication & Role-Based Dashboard

EduConnect must support three distinct user roles: Admin, Faculty, and Student. Although you are not required to implement a real login backend, you must simulate authentication using an in-memory user store and C# session state held in a scoped service (AuthStateService).

**Requirements:**

- Build a Login Page (/login) with a Bootstrap form. Fields: Email and Password (mock validation against in-memory data).
- On successful login, store the logged-in Person object in AuthStateService. Fire a login event (delegate/event) that other components can subscribe to.
- Implement a NavBar component that dynamically shows different menu items based on role (Admin sees everything, Faculty sees courses and grades, Student sees their own data).
- Protect routes: if a Student navigates to /admin, redirect them to /unauthorized. Use a custom AuthGuard component.
- Implement a Logout button that clears state and fires a logout event.

**Note:**

The NavBar must use one-way data binding to display the logged-in user's name and role badge. The AuthStateService must expose an event (Action or EventHandler) so that NavBar re-renders reactively when the user logs in or out.

### 2.2 Module 2: Student Management

The admin role manages all students. This module is the most data-intensive part of the project and requires careful use of collections, data storing and data binding.

**Requirements:**

- Student List Page (/admin/students): Displays all students in a Bootstrap table. Supports live search/filtering by name using two-way data binding (@bind on an input field). The displayed list must update in real time as the user types — no button click required.
- Add Student Page (/admin/students/add): A full Bootstrap form with fields for FullName, Email, Semester (1–8 as a dropdown), and CGPA. All fields must be validated using your IValidatable implementation. Show inline error messages below each invalid field.
- Edit Student Page (/admin/students/edit/{id}): Pre-populate the form with existing data (one-way binding on load; two-way binding for edits). On save, update the in-memory repository and fire an OnStudentUpdated event.

- Student Detail Page (/admin/students/{id}): Shows full profile including all enrollments, CGPA, and grade history. Use C# to compute average marks and sort courses alphabetically.
- Delete: Implement a confirmation modal (Bootstrap) before deletion. If student has active enrollments, throw and catch a custom StudentHasActiveEnrollmentsException and display an error — do not delete.

## 2.3 Module 3: Course Management & Enrollment

This module handles the full course lifecycle — from creation by admin to enrollment by Students. This is where your collection skills and OOP design are most prominently tested.

**Requirements:**

- Course Catalog Page (/courses): Viewable by all roles. Shows a Bootstrap card grid of all courses with Code, Title, CreditHours, and current enrollment count vs MaxCapacity. Use a computed property or method in the Course class to return EnrollmentStatus (enum: Open, AlmostFull, Full).
- Faculty can view their assigned courses only (/faculty/courses).
- Admin Course Management (/admin/courses): Full CRUD for courses. Implement using the ICourseService interface.
- Enrollment (/student/enroll): Students see available (not full) courses. Clicking Enroll calls ICourseService.EnrollStudent(). If the course is full, throw CourseFullException — catch it in the component and display an alert. On success, fire an OnEnrollmentChanged event that updates the NavBar badge showing the student's total enrolled credits.
- Drop Course: Students can drop a course only if its Status is Active. Dropped courses cannot be re-enrolled in the same semester — enforce this with a business rule in the service layer, not in the component.

## 2.4 Module 4: Grading System

This is the most algorithmically rich module. Faculty submit grades; Students view grades. CGPA is computed from grade records using C#.

**Requirements:**

- Grade Submission (/faculty/grades): Faculty select one of their assigned courses from a dropdown (two-way binding). The page then loads the enrolled students for that course (event-driven: OnCourseSelected EventCallback<Course>). Faculty enter marks (0–100) for each student in an inline-editable table. Validate marks using IValidatable. On Submit, call IGradeService.SubmitGrade() for each record and fire an OnGradesSubmitted event.
- Grade computation: Implement letter grade computation as a property in GradeRecord: A (85+), B (70–84), C (55–69), D (45–54), F (<45). CGPA uses a 4.0 scale: A=4.0, B=3.0, C=2.0, D=1.0, F=0.0, weighted by credit hours.
- Student Grade View (/student/grades): Shows a table of all enrolled courses with marks, letter grade, and status. A summary card at the top shows CGPA. Apply conditional CSS classes: green background for A/B, yellow for C/D, red for F.
- Admin Grade Report (/admin/reports/grades): Shows a summary table of all students with their CGPA, sorted descending. Includes a count of students in each grade category.

## 2.5 Module 5: Notification System

This is the event-driven centerpiece of EduConnect. The NotificationService must use C# delegates and events — not polling, not manual passing — to broadcast notifications across the application in real time.

**Requirements:**

- Design NotificationService with a public event: event Action<Notification> OnNewNotification. Any component that needs to react to new notifications subscribes to this event in OnInitialized() and unsubscribes in IDisposable.Dispose().
- Trigger notifications automatically when: a student is enrolled in a course (NotificationType: Enrollment), grades are submitted for a course the student is enrolled in (NotificationType: GradePosted), and an Admin sends a manual broadcast (NotificationType: Announcement).
- NotificationBell component: A reusable component in the NavBar showing a bell icon with a Bootstrap badge displaying unread count. Clicking it opens a dropdown list of the latest 5 notifications. This component subscribes to OnNewNotification and calls StateHasChanged() to re-render reactively.
- Notifications Page (/notifications): Full list of all notifications for the logged-in user. Mark as read functionality using IGradeService.
- The Admin Broadcast Page (/admin/notifications): A form with a message textarea (two-way binding) and a role selector checkbox group. On send, create notification objects for all users matching the selected roles and fire the event for each.

## 3. SOLID Principles-Mandatory Compliance

Every class you write in EduConnect will be evaluated against the SOLID principles. The following table shows exactly how each principle must be demonstrated in your project. Your code review (part of the submission) must include comments in your code explicitly identifying where each principle is applied.

| SOLID Principle | How Applied in EduConnect |
|---|---|
| **Single Responsibility (SRP)** | Each service class has exactly one reason to change. StudentService only manages student data. NotificationService only manages notifications. Components only handle UI rendering — business logic lives in services, never in .razor files. |
| **Open/Closed (OCP)** | The IRepository<T> interface allows new entity types to be added without modifying existing repository code. The Person hierarchy allows new user roles to be added (e.g., Librarian) without changing existing Student or Faculty logic. |
| **Liskov Substitution (LSP)** | Anywhere a Person is expected, you can substitute a Student, Faculty, or Admin without breaking behavior. The GetRole() method must behave correctly for each subtype. IRepository<Student> and IRepository<Course> must be interchangeable where IRepository<T> is expected. |
| **Interface Segregation (ISP)** | IStudentService extends IRepository<Student> but does NOT include grade-related methods — those belong to IGradeService. No class is forced to implement methods it does not need. IValidatable is a separate interface implemented only by classes that require validation. |
| **Dependency Inversion (DIP)** | All Blazor pages and components receive their dependencies (services) via constructor injection or [Inject] attribute — never by instantiating services with new(). Services are registered in Program.cs as interfaces. |

## 4. Blazor-Specific Requirements

This section details the exact Blazor features you must use. Simply building a static HTML page with no reactivity will not satisfy these requirements. EduConnect must demonstrate that you understand how Blazor's component model, data binding, and event system work.

### 4.1 Components (Minimum Required)

You must build at least the following reusable components (not pages — these are components embedded inside pages):

- AlertBox.razor — Accepts a Message (string) and Type (enum: Success, Warning, Error, Info) as parameters. Renders a Bootstrap alert with the appropriate color class. Used everywhere errors and confirmations appear.
- ConfirmDialog.razor — A Bootstrap modal with Title, Message, and two EventCallbacks: OnConfirmed and OnCancelled. Used before any destructive action (delete, drop course).
- StudentCard.razor — Displays a student summary card. Accepts a student parameter and shows name, semester, and CGPA with a color-coded badge.
- CourseCard.razor — Displays a course summary card. Accepts a Course parameter and an OnEnroll EventCallback and shows enrollment progress as a Bootstrap progress bar (calculated as enrolled/maxCapacity * 100).
- NotificationBell.razor — Described in Module 5 above.
- GradeTable.razor — Accepts a List<GradeRecord> parameter. Renders an interactive table with conditional row coloring. Used in both faculty grade submission and student grade view.
- LoadingSpinner.razor — Shows a Bootstrap spinner. Accepts an IsLoading (bool) parameter. Wrap async operations with this component.

### 4.2 Data Binding Requirements

You must explicitly demonstrate the following binding scenarios in your code:

- One-Way Binding: Student dashboard — display logged-in student's name, CGPA, and enrolled courses from the AuthStateService. These values are displayed, not edited, so use @FullName, @student.CGPA, etc. (expression binding).
- Two-Way Binding: Search input on the Student List page. The search string variable is bound with @bind="searchTerm" on the input, and the displayed list is filtered via a computed property that depends on searchTerm. Typing must instantly update the list — no submit button.
- Two-Way Binding on Forms: All Add/Edit forms must use @bind on each input. Marks entry in the grade table must use @bind on each input cell.
- EventCallback Binding: CourseCard must accept an OnEnroll EventCallback<Course>. When the Enroll button is clicked, the callback is invoked passing the Course object to the parent page — which then calls the service. This demonstrates parent-child component communication.
- Cascading Parameters: Use a CascadingValue in MainLayout to cascade the current AuthState down to child components, eliminating the need to pass it as a parameter to every component.

### 4.3 Routing Requirements

Your application must define and correctly implement the following routes using the @page directive:

- /login — Public
- /dashboard — Authenticated, role-specific content
- /admin/students — Admin only

- /admin/students/add — Admin only
- /admin/students/edit/{Id:guid} — Admin only, Id is a route parameter
- /admin/students/{Id:guid} — Admin only
- /admin/courses — Admin only
- /admin/reports/grades — Admin only
- /admin/notifications — Admin only
- /faculty/courses — Faculty only
- /faculty/grades — Faculty only
- /student/enroll — Student only
- /student/grades — Student only
- /courses — All authenticated roles
- /notifications — All authenticated roles
- /unauthorized — Public

## 5. Grading Rubric

Marks are distributed across the following components. Partial credit is only awarded when the feature is functionally correct — a feature that crashes or produces wrong output receives zero, regardless of how much code was written.

| Component | Weight | Description |
|---|---|---|
| Identification of correct classes | **20 marks** | Person hierarchy, all entity classes, all interfaces defined correctly, DI registration in Program.cs, folder structure |
| Student Management Module | **15 marks** | Full CRUD, live search (two-way binding), custom exceptions, IValidatable, filtering |
| Course & Enrollment Module | **15 marks** | Course catalog, enrollment/drop logic, CourseCard EventCallback, business rules enforced in service layer |
| Grading System | **15 marks** | Grade submission, letter grade computation, CGPA calculation, conditional CSS, Admin grade report |
| Notification System | **10 marks** | event Action<T>, all trigger points, NotificationBell reactive re-render, IDisposable unsubscription |
| SOLID Compliance | **10 marks** | All 5 principles demonstrable with comments, DI everywhere, no new() for services in components |
| Blazor UI/UX | **10 marks** | Bootstrap responsive layout, all required components built, no inline styles, mobile-friendly at 768px |
| Code Quality & Clean Code | **5 marks** | Meaningful names, no dead code, no God classes, no business logic in .razor files |

**Note:**

**Submit your own work. Your efforts will be valued but copy pasting will be awarded a straight ZERO in the assignment.**

**Good Luck!!!** ☺